

Agency Client & Campaign Tracker

Submitted by: Abhinav Jain | Registration No: 25BAI10234 | Course: Programming in Java

September 18, 2026

Contents

1. Introduction	2
2. Problem Statement	2
3. Objectives	2
4. Functional Requirements	2
5. Non-Functional Requirements	3
6. Design	4
6.1 Use Case Diagram	4
6.2 Class Diagram	4
6.3 Process Flow – Generate Invoice	6
7. Implementation Details	6
8. Testing	8
9. Challenges Faced	8
10. Learnings	8
11. Future Enhancements	9
12. References	9

1. Introduction

Small marketing and growth agencies typically manage several clients at once, each with campaigns running and invoices that need to be raised and followed up on. When this information is tracked informally – across spreadsheets, notes, or chat threads – it becomes easy to lose sight of which campaigns are active, which client owes what, and how much revenue has actually been collected versus is still outstanding. As the number of clients grows, this kind of ad-hoc tracking breaks down quickly.

This project implements a **command-line Java application** that gives agency staff a simple, menu-driven way to record clients, the campaigns run for each one, and the invoices raised against them – all from a terminal, with no database server, no internet connection, and no installation beyond a Java runtime. The entire application lives in a single file, `Main.java`, which was a deliberate choice to keep the project's scope and structure easy to read, explain, and maintain for a second-year course submission, while still demonstrating the core Java concepts the course covers: object-oriented design, collections, custom exception handling, and file I/O.

2. Problem Statement

Manually tracking clients, campaigns, and billing across notes or spreadsheets leads to lost context: there is no single place to check which campaigns are currently active, no consistent way to see how much money has been collected versus how much is still owed, and no safeguard against accidentally creating a campaign or invoice for a client that was never actually added. This project addresses that gap with a lightweight, file-based console tool that requires nothing more than a working JDK to run, and that persists its data locally so nothing has to be re-entered between sessions.

3. Objectives

The project set out to achieve the following:

1. Let agency staff record clients and view the full client list at any time.
2. Let staff create campaigns tied to a specific, existing client, and view all campaigns created so far.
3. Let staff generate invoices for a client and mark them as paid once payment is received.
4. Provide simple, at-a-glance reports: total clients, total campaigns, revenue collected, and outstanding dues.
5. Persist all of the above between runs using a local file, so the application remembers its state without needing a database.
6. Handle predictable bad input (an unknown client ID, non-numeric amounts) gracefully, without the program crashing.

4. Functional Requirements

#	Module	Description
FR1	Client Management	Add a client (name, industry, email); view all clients added so far.
FR2	Campaign Management	Create a campaign for an existing client (title, budget); view all campaigns.
FR3	Invoicing	Generate an invoice for a client; mark an invoice as paid; view all invoices and their status.
FR4	Reports	Show total clients, total campaigns, revenue collected, and outstanding dues.

Input/output structure. All input is taken through a `Scanner` reading standard input, driven by numbered text menus (e.g. “1. Client Management”, “2. Campaign Management”, ...). All output is printed as plain text lines to standard output – either a confirmation message (e.g. “Client added with ID: C1”), a formatted list of records, or an error message when something goes wrong.

Logical workflow. The application starts by loading any previously saved data, then repeatedly shows the main menu until the user chooses “Save & Exit”. Each of the four modules (Client Management, Campaign Management, Invoicing, Reports) is its own sub-menu reached from the main menu, and each returns control back to the main menu when the user chooses “Back”.

5. Non-Functional Requirements

Requirement	How it is addressed
Reliability	A custom <code>NotFoundException</code> is thrown and caught whenever a campaign or invoice is created for a client ID that doesn't exist, instead of the program crashing with an unhandled error. <code>NumberFormatException</code> is separately caught wherever a numeric amount is expected (campaign budget, invoice amount), so typing text there produces a clean message rather than a crash.
Usability	Every screen in the application follows the same numbered-menu pattern, so a user only has to learn one interaction style to use every module.

Requirement	How it is addressed
Maintainability	Each of the four modules has its own dedicated menu method (<code>clientMenu()</code> , <code>campaignMenu()</code> , <code>invoiceMenu()</code> , <code>reportsMenu()</code>), keeping related logic grouped together even though everything lives in one file.
Persistence	All in-memory data is written to <code>agency_data.txt</code> when the user exits, and read back in at startup, so no data entered in a previous session is ever lost.
Simplicity / Low Setup Cost	The project depends on nothing beyond the Java standard library and compiles and runs with two commands (<code>javac Main.java</code> and <code>java Main</code>), with no build tool, IDE project file, or external library required.

6. Design

The application is intentionally kept to a single top-level class, `Main`, with three small data classes – `Client`, `Campaign`, and `Invoice` – declared as `static` nested classes inside it, plus one custom exception class, `NotFoundException`, also nested inside `Main`. This was a deliberate design choice: rather than splitting the project across many files and packages (which would add structural overhead without adding real functionality at this scale), everything relevant sits in one place and can be read from top to bottom.

At runtime, all data lives in three `ArrayLists` (`clients`, `campaigns`, `invoices`) held as `static` fields on `Main`. These are populated once at startup (from `agency_data.txt`, if it exists) and are written back out to that same file whenever the user saves and exits.

6.1 Use Case Diagram

The diagram below shows the single actor – Agency Staff – and the four main actions available to them, corresponding directly to the four functional modules.

6.2 Class Diagram

The class diagram shows `Main` holding and managing the three data classes. `Campaign` and `Invoice` both reference a client by its ID rather than holding a direct object reference, which keeps the file-based save format simple (each record only needs to store an ID, not a nested object).

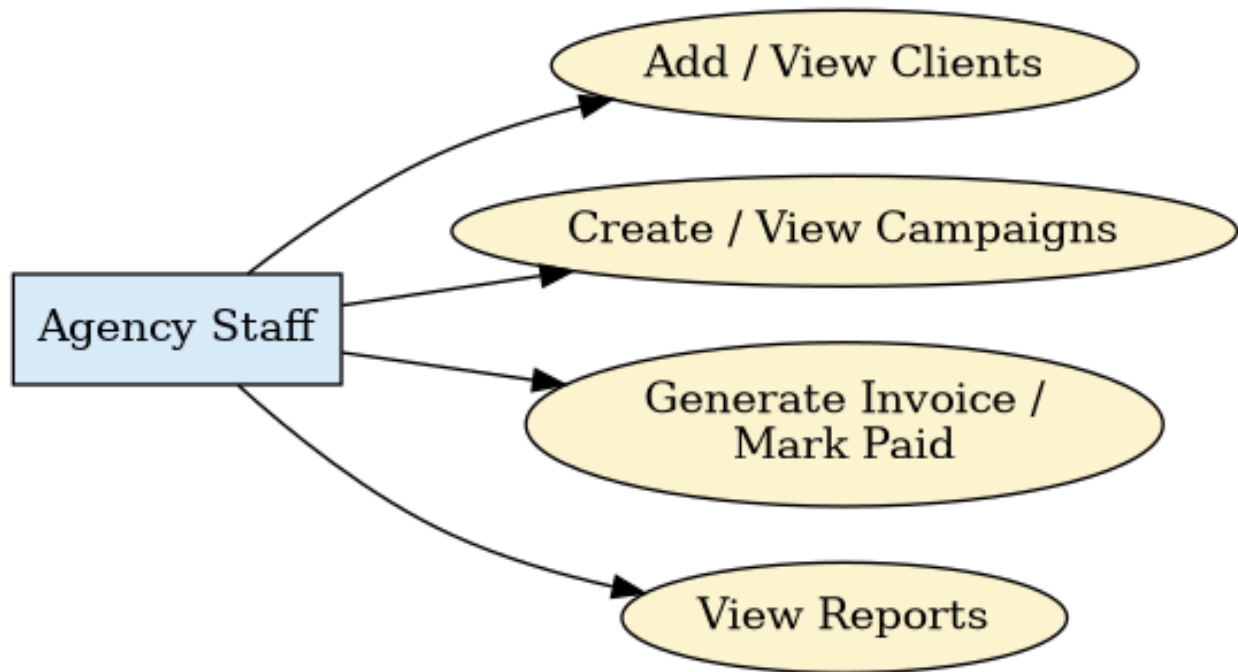


Figure 1: Use Case Diagram

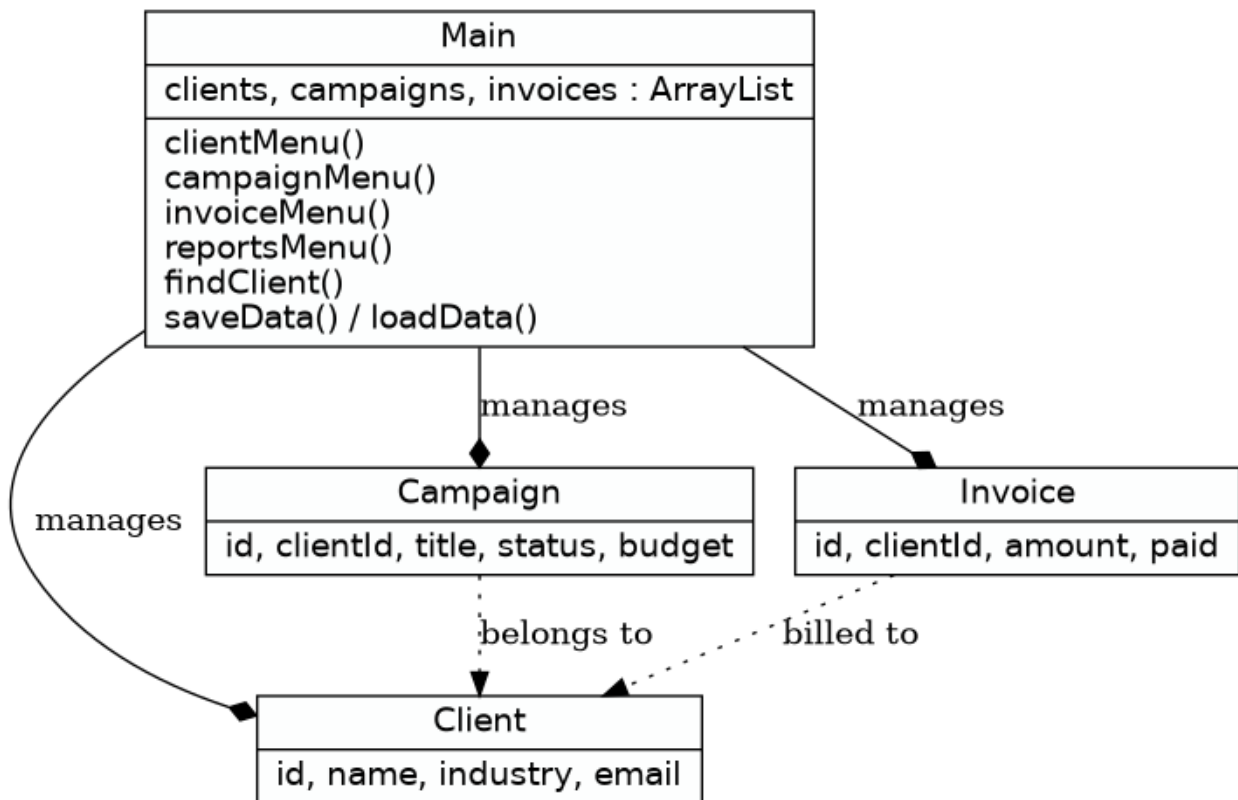


Figure 2: Class Diagram

6.3 Process Flow - Generate Invoice

The flow diagram below traces what happens when a staff member chooses “Generate Invoice”, including the branch where the client ID doesn’t exist and the custom exception is raised.

7. Implementation Details

- **Language and platform:** Java 17+, using only the standard library (java.io, java.util) – no external dependencies.
- **Data classes:**
 - Client – id, name, industry, email.
 - Campaign – id, clientId, title, status, budget.
 - Invoice – id, clientId, amount, paid.

Each class has a constructor that sets all its fields and a toString() override used to print it in list views.

- **In-memory storage:** three ArrayLists (clients, campaigns, invoices) declared as static fields on Main, alongside a Scanner for input and three integer counters used for ID generation.
- **ID generation:** simple incrementing counters (clientCounter, campaignCounter, invoiceCounter), each starting at 1, produce human-readable IDs like C1, CM1, and INV1. When data is reloaded from file, each counter is incremented once per matching record read, so new IDs continue from where the last session left off.
- **File format:** agency_data.txt stores one record per line, in comma-separated form, prefixed with a tag – CLIENT, CAMPAIGN, or INVOICE – that tells loadData() which kind of object to reconstruct from the rest of the line. For example:

```
CLIENT,C1,Sample Bakery Co.,Food & Beverage,hello@samplebakery.example  
CAMPAIGN,CM1,C1,Festive Season Promotion,ACTIVE,25000.0  
INVOICE,INV1,C1,12000.0,true
```
- **Error handling:** findClient(id) searches the clients list and either returns the matching Client or throws NotFoundException with a descriptive message. This is called before a campaign or invoice is created, so no campaign or invoice can ever reference a client that doesn’t exist. The calling menu method catches this exception (and separately catches NumberFormatException around numeric input) and prints a plain-language error instead of letting the program terminate.
- **Menu system:** the main loop and every sub-menu follow the same pattern – print a numbered list of options, read a line of input, and use a switch (or, for two-option sub-menus, simple if/else if) statement to dispatch to the right action.

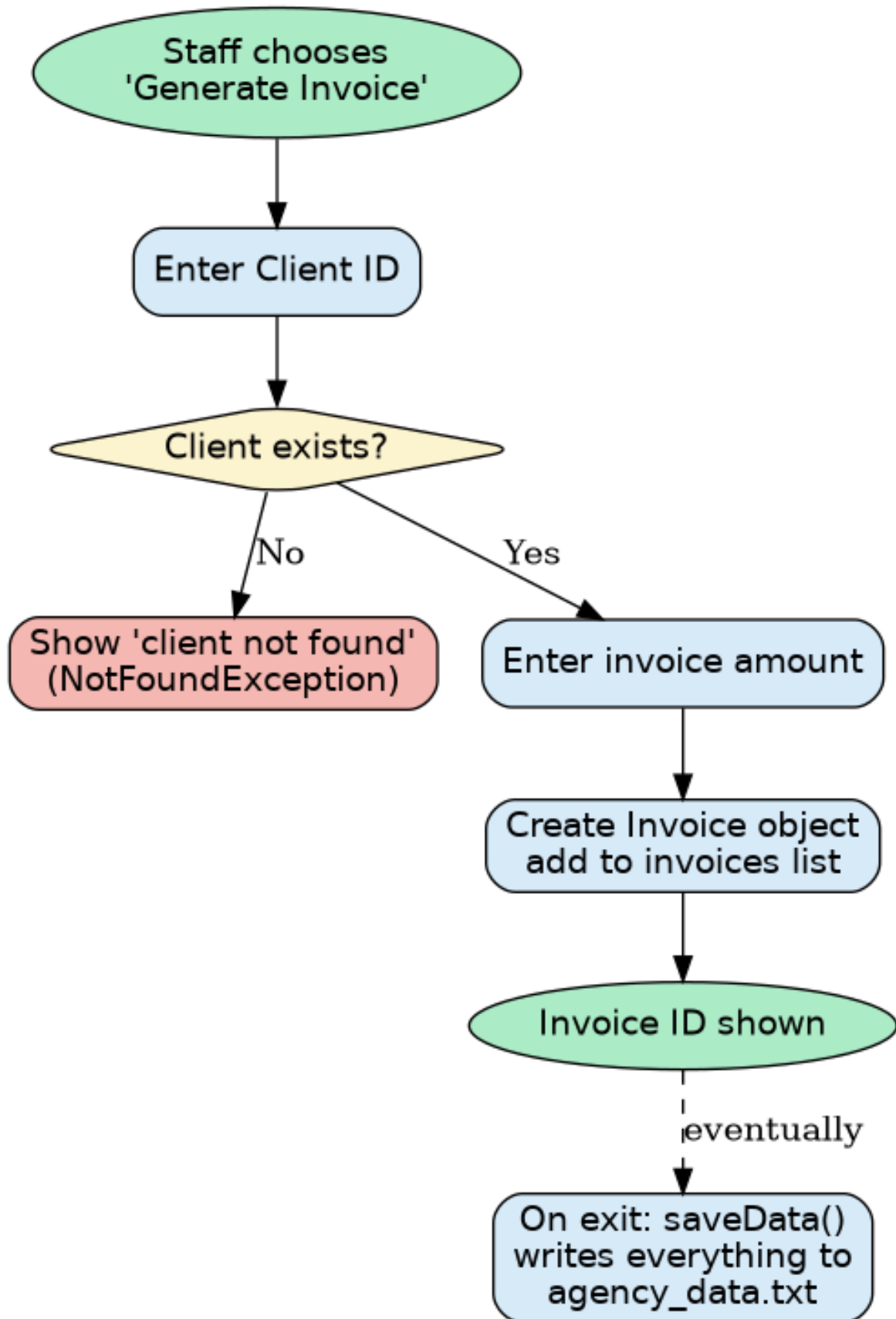


Figure 3: Generate Invoice Flow

8. Testing

The application was tested manually by walking through the complete intended workflow end to end:

1. Adding a client and confirming it appears correctly in the “View All Clients” listing.
2. Creating a campaign against that client and confirming it appears correctly in “View All Campaigns”, with the correct client ID attached.
3. Generating an invoice for the client, confirming it initially shows as UNPAID, then marking it paid and confirming the status updates to PAID.
4. Checking the Reports screen before and after marking the invoice paid, to confirm “Revenue collected” and “Outstanding dues” update correctly.
5. Deliberately entering a client ID that does not exist when creating a campaign and when generating an invoice, to confirm `NotFoundException` is caught and a clean error message is shown rather than a stack trace or crash.
6. Deliberately entering non-numeric text where a budget or invoice amount was expected, to confirm `NumberFormatException` is caught in the same way.
7. Saving and exiting, then restarting the application, to confirm every previously entered client, campaign, and invoice reloads correctly from `agency_data.txt` with the right values.

All of the above behaved as expected during testing.

9. Challenges Faced

- **Choosing a simple but reliable file format.** Since the project deliberately avoids a database or a serialization library, a plain-text format had to be designed by hand. The solution was to prefix every line with a record-type tag (`CLIENT`, `CAMPAIGN`, or `INVOICE`) so that a single loader method could correctly distinguish between the three kinds of records sharing one file.
- **Preventing “orphaned” campaigns or invoices.** Without a database’s foreign-key constraints, nothing would normally stop a campaign or invoice from being created against a client ID that doesn’t exist. This was solved with a single small `findClient()` helper, called before any campaign or invoice is created, that throws `NotFoundException` if the ID isn’t found – keeping the safeguard in one place rather than duplicating validation logic.
- **Keeping the design appropriately scoped.** An earlier version of this project split the same functionality across multiple files, packages, and a generic storage abstraction. That added real complexity without adding value at this project’s scale, so the final design was deliberately consolidated into a single, readable file – a conscious trade-off between “impressive-looking architecture” and “code I can confidently read, test, and explain.”

10. Learnings

- Practiced modeling real-world entities (`Client`, `Campaign`, `Invoice`) as classes with constructors and a clear `toString()` representation.

- Practiced designing and using a custom checked exception (`NotFoundException`) to make a specific failure mode explicit, rather than relying on `null` checks or letting the program crash.
- Practiced reading and writing structured, multi-type data to a single text file using `BufferedReader` and `BufferedWriter`, including designing a simple tagged-line format to tell different record types apart.
- Learned to weigh design complexity against project scope – that a single well-organized file can be a better fit than a multi-file architecture when the project’s size doesn’t call for it.

11. Future Enhancements

- Add a login step so only authorized staff can access the application.
- Support editing or deleting existing clients, campaigns, and invoices.
- Add due dates to invoices and automatically flag ones that have gone unpaid past their due date.
- Escape or validate commas in user-entered text (names, emails) so they can’t be typed in a way that would corrupt a saved record.
- Add basic input validation (e.g. rejecting an empty client name) beyond the numeric checks already in place.

12. References

1. Oracle, *Java Platform, Standard Edition Documentation* – <https://docs.oracle.com/en/java/javase/>
2. Oracle, *java.io Package Documentation* – <https://docs.oracle.com/en/java/javase/17/docs/api/java.io/package-summary.html>
3. Oracle, *java.util.ArrayList Documentation* – <https://docs.oracle.com/en/java/javase/17/docs/api/java.util/ArrayList.html>
4. Course materials for *Programming in Java* (flipped-course evaluation instructions document).